



CSL303/MAL505 — Database Management Systems

Module II: Database Design

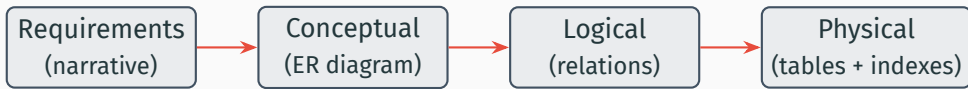
Lecture 13: The Entity-Relationship Model

Amit Kumar Dhar

IIT Bhilai

- **Weeks 1–4:** We could **use** a database — DDL, queries, transactions.
- But every query assumed the schema **already existed**.
- Who decides what tables to create, and why?
- **Module II — Database Design:** Turning a vague, human description of a problem into a correct, non-redundant relational schema.

The Design Pipeline



- **Conceptual** — What data exists, independent of any DBMS (this week).
- **Logical** — Relations, keys, FDs; normalization (Week 6).
- **Physical** — Files, indexes, storage (Weeks 7+).
- Today: The **Entity-Relationship (ER)** model — the standard conceptual tool.

Agenda

- Why a conceptual model
- Entities and entity sets
- Attributes and their kinds
- Keys in the ER world
- Relationships and relationship sets
- Mapping cardinalities (1:1, 1:N, M:N)
- Participation constraints (total/partial)
- Weak entity sets
- Drawing ER diagrams (Chen & crow's-foot)
- A worked mini-world

Why Model Before Building?

- Tables are **cheap to write** but **expensive to get wrong**.
- A bad schema causes redundancy, update anomalies, and impossible queries.
- A diagram is a **shared language** between you, the client, and teammates.
- Cheaper to erase a box on a whiteboard than to migrate a live database.
- ER modelling forces the key questions:

What are the things? How do they relate?

The Three Ingredients

An ER model describes a mini-world with just **three** concepts:

1. **Entities** — The “things” (a student, a course, a car).
2. **Attributes** — The properties of those things (a name, a price).
3. **Relationships** — How things are associated (a student *takes* a course).

Everything else in ER modeling is a refinement of these three.

Quick Check: Entities vs. Attributes

Question

Should **Department** be modeled as an attribute of **Student** (e.g., a `dept_name` text string) or as its own entity set? Why?

Quick Check: Entities vs. Attributes

Question

Should **Department** be modeled as an attribute of **Student** (e.g., a `dept_name` text string) or as its own entity set? Why?

Answer

- If we only need to store a simple string name and nothing else, an attribute might suffice.
- However, if a department has its own attributes (e.g., **building**, **budget**) or participates in other relationships (e.g., *offers* courses, *employs* instructors), **Department MUST be an entity set!**

Entities and Entity Sets

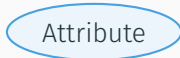
- An **entity** is a distinguishable object in the mini-world.
 - *e.g.*, the specific student “Aditi, ID 22001”.
- An **entity set** is a collection of entities of the **same type**.
 - *e.g.*, all **students**.
- In diagrams we draw the **set**, not individual entities.
- An entity set becomes (roughly) a **table**; an entity becomes a **row**.

Chen Notation — Core Shapes



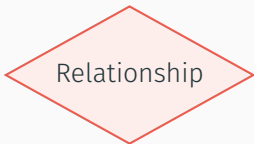
Entity Set

: Rectangle



Attribute

: Ellipse



Relationship

: Diamond



Weak Entity

: Double Rectangle



Ident. Rel.

: Double Diamond

- We use **Chen notation** to teach concepts, then **crow's-foot** for practice — both convey the same structural semantics.

- An **attribute** is a property mapping an entity to a value from a **domain**.
- **student** has attributes **id**, **name**, **tot_cred**.
- Each attribute has a **domain** (the set of allowed values): **INTEGER**, **TEXT**...
- Drawn as an ellipse attached to the entity's rectangle.

Kinds of Attributes (Part 1)

- **Simple** — Atomic, indivisible (`tot_cred`).
- **Composite** — Decomposes into parts (`name` → `first_name`, `last_name`; `address` → `street`, `city`, `pin`).
- **Single-valued** — One value per entity (`birth_date`).
- **Multivalued** — A set of values (`phone_numbers`) — drawn as a **double ellipse**.

Kinds of Attributes (Part 2)

- **Derived** — Computable from other attributes (**age** derived from **birth_date**) — drawn as a **dashed ellipse**. Store the base, derive the rest.
- **Stored** — Actually kept in the database (the base attribute).
- **Null** — Value missing, unknown, or not applicable.
- Composite + multivalued attributes **cannot** be stored directly in a plain relational column — mapping them out is next lecture's job.

Keys in the ER Model

- A **superkey** — A set of attributes that uniquely identifies an entity.
- A **candidate key** — A minimal superkey.
- The **primary key** — The candidate key we choose to identify entities.
- Underlined in the diagram: id.
- Every (strong) entity set needs a primary key — otherwise its entities are indistinguishable.

Choosing a Good Primary Key

- **Stable** — Doesn't change over an entity's lifetime (not `email`).
- **Minimal** — As few attributes as necessary.
- **Never null** — Identity requires a value.
- **Natural keys** (a real-world ID) vs. **surrogate keys** (a system-generated number).
- When no natural key is stable, **introduce a surrogate** (`student_id`).

Quick Check: Attribute Types & Keys

Question

Suppose we want to record a student's **age** and **email_addresses** (a student can have multiple emails).

1. How are these represented in Chen ER notation?
2. Can **email** serve as the Primary Key for **Student**?

Quick Check: Attribute Types & Keys

Question

Suppose we want to record a student's **age** and **email_addresses** (a student can have multiple emails).

1. How are these represented in Chen ER notation?
2. Can **email** serve as the Primary Key for **Student**?

Answer

1. **age** is a **derived attribute** → **Dashed Ellipse** (derived from **birth_date**).
email_addresses is a **multivalued attribute** → **Double Ellipse**.
2. **No.** **email** is multivalued (a student may have multiple emails) and can change over time. Primary keys must be single-valued, stable, and unique. A surrogate key like **student_id** is preferred.

Relationships

- A **relationship** associates two (or more) entities.
 - “Aditi **takes** CS-101.”
- A **relationship set** is a set of relationships of the same type.
- Drawn as a **diamond** linking the participating entity sets.
- The entity sets are the relationship’s **participants**; each participant plays a **role**.

Degree of a Relationship

- **Binary** — Two participants (by far the most common): **student** *takes* **course**.
- **Ternary** — Three participants: **supplier** *supplies* **part** to **project**.
- **Recursive (unary)** — An entity set related to **itself** with explicit **roles**:
 - **employee** *manages* **employee** (roles: *manager* / *subordinate*).
- Higher-degree relationships are real but rarer; watch for hidden binary decompositions.

Relationship Attributes

- A relationship can carry its **own** attributes.
- **takes** has a **grade** — it belongs to neither the student **nor** the course alone, but to the **pairing**.
- Drawn as an ellipse attached to the relationship's diamond.
- These attributes become columns of the relationship's table during logical schema mapping.

Mapping Cardinalities

- Expresses how many entities on one side may relate to one entity on the other side.
- Three canonical cases for binary relationships:
 - **One-to-one (1:1)**
 - **One-to-many (1:N)**
 - **Many-to-many (M:N)**
- This is the single most important constraint to get right — it directly drives foreign key placement and junction tables.

One-to-One (1:1) Relationships

- Each entity on side A relates to **at most one** entity on side B, and vice versa.
- *e.g.*, **employee manages department** (each department has one manager; each manager heads at most one department).



One-to-Many (1:N) Relationships

- One entity in A relates to **many** entities in B, but each entity in B relates to **at most one** entity in A.
- e.g., one **instructor** **advises** many **students**; each student has one advisor.
- The “many” side is where the foreign key will live in the database tables.



Many-to-Many (M:N) Relationships

- One entity in A relates to many in B **and** one entity in B relates to many in A.
- *e.g.*, a **student takes** many **courses**; a **course** is taken by many **students**.
- M:N relationships **always** become their own table (a junction/associative table).



Reading Cardinality from Requirements

- Listen carefully for quantifier phrasing in problem statements:
 - “Each order has exactly one customer” → Orders to Customer is **N:1**.
 - “A student may enroll in several sections” → **M:N**.
 - “Every account belongs to one branch” → **N:1**.
- Always ask the client questions in **both directions**:

“Can a course have many students? Can a student have many courses?”

Participation Constraints

- Specifies whether **every** entity in an entity set must participate in the relationship.
- **Total participation** (drawn as a **double line**) — Every entity **must** participate in at least one relationship.
 - *e.g.*, every **loan** must be held by at least one **customer**.
- **Partial participation** (drawn as a **single line**) — Participation is optional.
 - *e.g.*, not every **customer** has a **loan**.

Cardinality + Participation Together

- The two constraints are **independent** and combine to fully constrain relationships.
- “Each section **must** have exactly one instructor” = Instructor side: **total**, cardinality **1**.
- “A student **may** have an advisor” = Partial participation on the student side.
- Together they specify the **minimum and maximum** structural constraints on each side.

(min, max) Notation

- An alternative notation: Label each edge with an explicit (min, max) pair.
- `student (1,1) --- advisor --- (0,*) instructor`:
 - Each student has **exactly one** advisor (1,1).
 - Each instructor advises **zero or more** students (0,*).
- Highly precise and unambiguous; preferred in some textbook standards over arrow notations.

Quick Check: Mapping Cardinalities

Question

In a corporate database requirements text:

1. An employee works in at most one department, but a department employs many employees. Every employee must belong to a department.
2. An employee can lead at most one project, and a project may or may not have a leader assigned.

What are the cardinalities and participation constraints for **works_in** and **leads**?

Quick Check: Mapping Cardinalities

Question

In a corporate database requirements text:

1. An employee works in at most one department, but a department employs many employees. Every employee must belong to a department.
2. An employee can lead at most one project, and a project may or may not have a leader assigned.

What are the cardinalities and participation constraints for **works_in** and **leads**?

Answer

1. **works_in**: **N:1** (Employee to Department). Employee participation is **Total** (double line); Department participation is **Partial**.
2. **leads**: **1:1** (Employee to Project). Both sides have **Partial** participation.

Weak Entity Sets

- A **weak entity** has no primary key of its own — it cannot be uniquely identified without an owner entity.
- Classic examples: **section** depends on **course**; a **payment** depends on a **loan**.
- Drawn as a **double rectangle**.
- Identified via an **identifying relationship** (**double diamond**) connected to its **owner** (strong entity set).



Discriminator (Partial Key)

- A weak entity set has a **discriminator** (or partial key) — a set of attributes that distinguishes entities **belonging to the same owner entity**.
- **section** discriminator: (**sec_id**, **semester**, **year**) — unique **within a specific course**, but not globally.
- Drawn with a **dashed underline** in Chen notation.
- **Full Primary Key** of a weak entity set = **Owner's Primary Key + Discriminator**.

When is it Really Weak?

- Weak Entity $\Leftrightarrow$ Existence **depends** on owner **AND** it borrows the owner's key for identity.
- If you assign it a unique, stable surrogate key (e.g., `section_id` UUID), it becomes a **strong** entity with a mandatory relationship instead.
- Both modeling choices can be valid — document your rationale and maintain consistency.

Quick Check: Weak Entities

Question

Why can't `sec_id` (e.g., "Section 1") serve as the primary key for **Section** on its own? What constitutes the complete primary key for **Section**?

Quick Check: Weak Entities

Question

Why can't `sec_id` (e.g., “Section 1”) serve as the primary key for **Section** on its own? What constitutes the complete primary key for **Section**?

Answer

- `sec_id` is only a **discriminator (partial key)** because multiple distinct courses (e.g., CS101, CS303) have a “Section 1”.
- The complete primary key is:

(course_id, sec_id, semester, year)

which combines the owner entity's primary key (`course_id`) with the discriminator attributes!

A Mini-World: The University (Informal)

*“The university has **departments**, each with a name, building, and budget. **Instructors** and **students** each belong to a department. Instructors **advise** students. **Courses** are offered by departments and run as **sections** in a given semester. Students **enroll** in sections and receive a grade. Some courses require **prerequisite** courses.”*

Let's turn each phrase into ER structural components step by step.

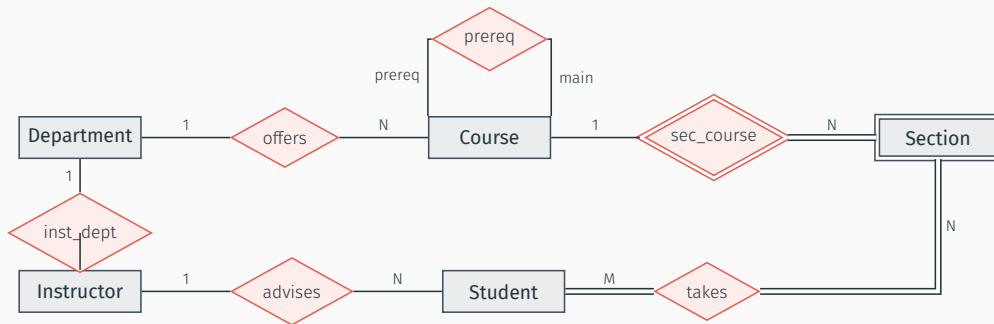
Extracting Entities and Attributes

- Nouns that represent standalone things → Entity Sets:
 - department, instructor, student, course, section.
- Nouns that represent descriptive properties → Attributes:
 - Department's dept_name, building, budget.
 - Student's ID, name, tot_cred.
 - Instructor's ID, name, salary.
 - Course's course_id, title, credits.
- Nouns/verbs describing associations → **Relationships** (next slide).

Extracting Relationships

- Verbs connecting entities → Relationship Sets:
 - Instructor **advises** student (1:N).
 - Course is **offered_by** department (N:1).
 - Instructor **inst_dept** belongs to department (N:1).
 - Student **stud_dept** belongs to department (N:1).
 - Student **takes** section (M:N, carrying relationship attribute **grade**).
 - Course **prereq** requires course (M:N, recursive prerequisite).
- **section** is a **weak entity set**, identified by **sec_course** from **course**.

The University ER Diagram



Double lines indicate total participation / identifying relationship.

Crow's-Foot Notation

- Industry standard notation used by modern ER tools (e.g., dbdiagram.io, draw.io, ERDplus).

- Cardinality and participation are shown directly by **line endings**:

`|` = ``exactly one''      `o` = ``zero''

`<` = ``many'' (the crow's foot)

- `student }o--|| advisor` reads: *“many students relate to exactly one advisor”*.
- Same underlying semantics as Chen notation; more compact because relationships are drawn as lines rather than diamond boxes.

Common Modelling Mistakes

- **Making an attribute what should be an entity:**
 - *e.g.*, storing **department** as a plain text string instead of its own entity set (when you eventually need building & budget).
- **Making an entity what is really an attribute:**
 - Creating a separate entity set for **color** when it is just a lone string attribute.
- **Missing a relationship attribute:**
 - Shoving **grade** into **Student** or **Course** instead of on the **takes** relationship.
- **Forgetting cardinality:**
 - Leaving relationships unconstrained silently permits invalid data states.

Entity vs. Attribute — The Test

- Ask: Does this item have its own attributes or participate in other relationships?
 - Yes → Make it an **Entity Set**.
 - No, it is just a single atomic value → Keep it an **Attribute**.
- **department** has a building, budget, and offers courses → **Entity Set**.
- **gender** is a single value → **Attribute**.

ER is a Modelling Aid, Not a Straitjacket

- Different database designers can draw different (yet equally valid) ER diagrams for the exact same real-world scenario.
- **Goal:** Capture all real-world constraints faithfully while avoiding structural redundancy.
- We design on paper, deliberate over cardinalities, and iterate before committing.
- **Next Lecture:** We will convert any valid ER diagram into concrete relational tables using a precise mechanical mapping algorithm.

Summary — Key Takeaways

- ER models a mini-world using **entities**, **attributes**, and **relationships**.
- Attributes may be simple/composite, single/multivalued, stored/derived.
- **Mapping cardinality** (1:1, 1:N, M:N) and **participation** (total/partial) are the core constraints shaping the schema.
- **Weak entity sets** borrow their owner's primary key + a local discriminator.
- Get the conceptual ER diagram right before writing a single **CREATE TABLE** statement!

Exercises

1. Model a **library** mini-world: books, physical copies, members, loans. Identify any weak entity sets.
2. In a **hospital** system, determine the cardinality and participation for **doctor** *treats* **patient**.
3. Draw an ER diagram for a **flight / passenger / booking** system where **seat_no** is a relationship attribute.
4. Give a real-world scenario where the same fact can be modeled as either an attribute or an entity set — and argue which design is superior.

Quick Reference — ER Symbols

Symbol	Meaning
Rectangle	Entity set (Double = Weak entity set)
Ellipse	Attribute (Double = Multivalued, Dashed = Derived)
Diamond	Relationship set (Double = Identifying relationship)
<u>Underline</u>	Primary key attribute (Dashed underline = Discriminator)
1 / N / arrow	Mapping cardinality
Double line	Total participation constraint

Before Next Lecture

- Try drawing the library ER diagram (on paper or using any online tool).
- **Optional reading:** Silberschatz §6.1–§6.5 (The ER Model).

Next Time — Lecture 14

EER features (generalization, specialization, aggregation) and the step-by-step algorithm to map ER diagrams to relational schemas.

Questions?